

Product Listing with Search & Filter

Assignment 6 — Dynamic Product Listing with `ListView.builder` & `setState`

Name: Soumitra Deshpande

Roll No: 150096724035

Cohort: Elon Musk

Environment: Flutter 3.x / Dart 3.x

Code: https://github.com/SoumitraDeshpande11/assignments/tree/assignments_6

1. Objective

Build a dynamic product listing app in Flutter. Products must be described by a **data model class**, rendered with an efficient `ListView.builder`, and the user must be able to **search by name** and **filter by category**, with every change applied through `setState`.

2. Requirements Checklist

Requirement	Where it lives
Data model class	Product — an immutable class with name, category, price, rating, and emoji fields
<code>ListView.builder</code>	Builds one card per product in the filtered list, lazily, from <code>itemCount: filtered.length</code>
Search	<code>TextField</code> whose <code>onChanged</code> stores the query in state via <code>setState</code>
Filter	A horizontal row of <code>ChoiceChips</code> (All, Electronics, Clothing, Grocery, Books) selected via <code>setState</code>
<code>setState</code>	<code>_onSearchChanged</code> , <code>_onCategorySelected</code> , and <code>_clearSearch</code> all wrap their changes in <code>setState</code>

3. How the Data Model Works

Each product is a `Product` object:

```
class Product {  
  const Product({required this.name, required this.category,  
    required this.price, required this.rating, required this.emoji});  
  final String name;  
  final String category;  
  final double price;  
  final double rating;
```

```

    final String emoji;
}

```

The catalog is a hard-coded list of 12 Product instances stored in the State class. Because the fields are `final`, products never change — only **which** products are shown changes.

4. How Search & Filter Work

Two pieces of state drive the UI:

- `_searchQuery` — the current text in the search box.
- `_selectedCategory` — the currently selected chip ('All' by default).

Both are combined in a getter that recomputes the visible list on every rebuild:

```

List<Product> get _filteredProducts {
  final query = _searchQuery.toLowerCase();
  return _products.where((p) {
    final matchesSearch =
      query.isEmpty || p.name.toLowerCase().contains(query);
    final matchesCategory =
      _selectedCategory == 'All' || p.category == _selectedCategory;
    return matchesSearch && matchesCategory;
  }).toList();
}

```

Typing in the search box or tapping a chip calls `setState`, Flutter rebuilds the widget tree, the getter runs again, and `ListView.builder` shows only the matching products. A live counter above the list shows "X of 12 products", and a friendly message appears when nothing matches.

5. Widget Map

```

MaterialApp
├─ Scaffold
│   ├─ AppBar ('Product Catalog')
│   └─ Column
│       ├─ TextField                ← search box (with clear button)
│       ├─ ListView.separated      ← horizontal row of ChoiceChips
│       ├─ Text                    ← "X of 12 products" counter
│       └─ Expanded
│           └─ ListView.builder
│               └─ Card per product
│                   └─ ListTile[ emoji avatar, name, rating · category,
price ]

```

6. Source Code

```

lib/main.dart

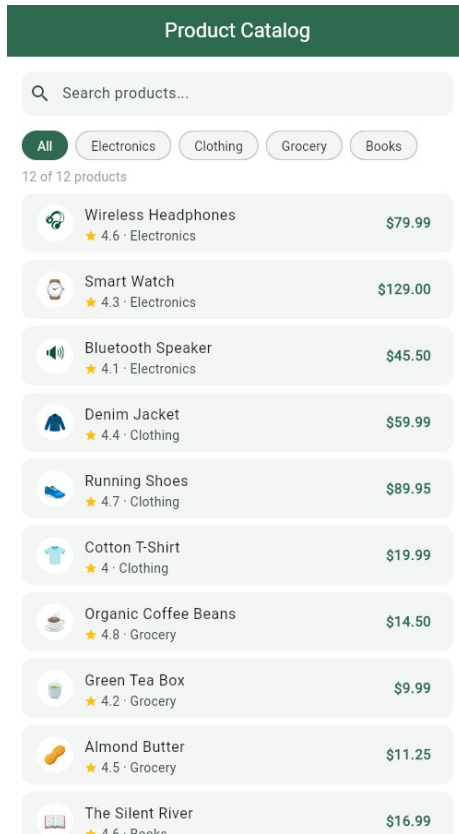
1  import 'package:flutter/material.dart';
2
3  void main() => runApp(const ProductApp());
4
5  class Product {
6    const Product({
7      required this.name,
8      required this.category,
9      required this.price,
10     required this.rating,
11     required this.emoji,
12   });
13
14   final String name;
15   final String category;
16   final double price;
17   final double rating;
18   final String emoji;
19 }
20
21 class ProductApp extends StatelessWidget {
22   const ProductApp({super.key});
23
24   @override
25   Widget build(BuildContext context) {
26     return MaterialApp(
27       title: 'Product Catalog',
28       debugShowCheckedModeBanner: false,
29       theme: ThemeData(
30         useMaterial3: true,
31         scaffoldBackgroundColor: Colors.white,
32         colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF2D6A4F)),
33       ),
34       home: const CatalogScreen(),
35     );
36   }
37 }
38
39 class CatalogScreen extends StatefulWidget {
40   const CatalogScreen({super.key});
41
42   @override
43   State<CatalogScreen> createState() => _CatalogScreenState();
44 }
45
46 class _CatalogScreenState extends State<CatalogScreen> {
47   static const String _all = 'All';
48
49   final TextEditingController _searchController = TextEditingController();
50
51   final List<Product> _products = const [
52     Product(name: 'Wireless Headphones', category: 'Electronics', price: 79.99, rating: 4.6, emoji: '🎧'),
53     Product(name: 'Smart Watch', category: 'Electronics', price: 129.00, rating: 4.3, emoji: '⌚'),
54     Product(name: 'Bluetooth Speaker', category: 'Electronics', price: 45.50, rating: 4.1, emoji: '🔊'),
55     Product(name: 'Denim Jacket', category: 'Clothing', price: 59.99, rating: 4.4, emoji: '👕'),
56     Product(name: 'Running Shoes', category: 'Clothing', price: 89.95, rating: 4.7, emoji: '👟'),
57     Product(name: 'Cotton T-Shirt', category: 'Clothing', price: 19.99, rating: 4.0, emoji: '👕'),
58     Product(name: 'Organic Coffee Beans', category: 'Grocery', price: 14.50, rating: 4.8, emoji: '☕'),
59     Product(name: 'Green Tea Box', category: 'Grocery', price: 9.99, rating: 4.2, emoji: '🍵')
60   ];

```

7. App Screenshots

The app was built for the web (`flutter build web`) and captured in Chrome at two window sizes.

Phone width (500px):



Desktop width (1280px):

Product Catalog		
 Search products...		
All Electronics Clothing Grocery Books		
12 of 12 products		
	Wireless Headphones ★ 4.6 · Electronics	\$79.99
	Smart Watch ★ 4.3 · Electronics	\$129.00
	Bluetooth Speaker ★ 4.1 · Electronics	\$45.50
	Denim Jacket ★ 4.4 · Clothing	\$59.99
	Running Shoes ★ 4.7 · Clothing	\$89.95
	Cotton T-Shirt ★ 4 · Clothing	\$19.99
	Organic Coffee Beans ★ 4.8 · Grocery	\$14.50
	Green Tea Box ★ 4.2 · Grocery	\$9.99

8. Terminal

Real build-and-run session: `flutter pub get`, `flutter analyze` (no issues), `flutter build web`, then `flutter run -d web-server` to serve the app locally.

```
zsh - assignment_6

$ flutter --version
Flutter 3.47.2 • channel stable • https://github.com/flutter/flutter.git
Framework • revision d3b14c8769 (3 weeks ago) • 2026-08-26 16:07:51 -0700
Engine • hash 1cf1c4773fb941c4c74a7f8bb144a8837596c0f4 (revision a804b26164)
Tools • Dart 3.13.2 • DevTools 2.60.0

$ flutter pub get
Got dependencies!

$ flutter analyze
No issues found! (ran in 3.1s)

$ flutter build web --release
Compiling lib/main.dart for the Web... 19.6s
✓ Built build/web

$ flutter run -d web-server --web-port=8742
Launching lib/main.dart on Web Server in debug mode...
Waiting for connection from debug service on Web Server... 11.2s
lib/main.dart is being served at http://localhost:8742

Flutter run key commands.
r Hot reload.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).
```

9. Run Instructions

```
flutter pub get
```

```
flutter run          # pick a device, or use flutter run -d chrome
```

Type in the search box to narrow products by name, tap a category chip to filter, and tap the × in the search box to clear. Both filters combine — searching "tea" with the Grocery chip selected shows only grocery items matching "tea".

10. Possible Extensions

- Sort the list by price or rating with a dropdown.
- Load products from a JSON file or a real REST API instead of a hard-coded list.
- Add a detail page that opens when a product card is tapped.

Built by Soumitra Deshpande • Roll No. 150096724035 • Cohort: Elon Musk